

Manuale del Progettista

Progetto **Ruggine**

Corso di Programmazione di Sistema
2025/2026

Gruppo 50

Alessandro Annecchini s343760
Riccardo Marconi s342227
Francesco Palmisani s343429
Giosuè Pinto s342711

21 febbraio 2026

Indice

1 Introduzione **1**

2 Analisi schematica dei requisiti **2**
2.1 Obiettivo del sistema . . . 2
2.2 Requisiti funzionali . . . 2
2.3 Requisiti non funzionali . . . 3

3 Frontend **4**
3.1 Struttura interna . . . 4
3.1.1 ChatPage (ChatPage.jsx) . . . 4
3.1.2 InvitesPage (InvitesPage.jsx) . . . 5
3.1.3 LoginPage (LoginPage.jsx) . . . 6
3.2 Comunicazione e flussi . . . 6
3.3 Context . . . 7
3.4 Sicurezza e validazione . . . 8
3.5 Motivazioni progettuali . . . 8

4 Backend **9**
4.1 Struttura interna . . . 9
4.2 Flusso Logico del login . . . 10
4.3 Flusso Logico dei messaggi . . . 11
4.4 Main e Entry Point . . . 13
4.4.1 Main (main.rs) . . . 13
4.4.2 CPU logger (cpu_logger.rs) . . . 13
4.4.3 Error (error.rs) . . . 14
4.4.4 Router (router.rs) . . . 14
4.4.5 State (state.rs) . . . 15
4.5 Controllers . . . 15
4.5.1 Authentication Controller (authentication_controller.rs) . . . 15
4.5.2 Web socket Controller (web_socket_controller.rs) . . . 15
4.6 Services . . . 17
4.6.1 User Service (user_service.rs) . . . 17
4.6.2 Chat Service (chat_service.rs) . . . 17
4.6.3 Authentication Service (auth_service.rs) . . . 19
4.7 Repositories . . . 19
4.7.1 Group Repository (group_repository.rs) . . . 20
4.7.2 Invitation Repository (invitation_repository.rs) . . . 21
4.7.3 Message Repository (message_repository.rs) . . . 23

4.7.4 User Repository (user_repository.rs) 23

5 Database **25**
5.1 Struttura e tabelle principali 25
5.1.1 users 25
5.1.2 groups 26
5.1.3 group_members 27
5.1.4 messages 27
5.1.5 invitations 28
5.2 Tecnologie e motivazioni 28
5.3 Sicurezza e integrità 28

6 Proposte di migliorie e feature future **30**
6.1 Migliorie architetturali 30
6.2 Nuove feature 30
6.3 Riflessioni su Token e Sicurezza 31

Capitolo 1

Introduzione

Il progetto Ruggine adotta un'architettura a 3 tier (livelli), una delle soluzioni più diffuse e robuste per la realizzazione di applicazioni moderne, scalabili e manutenibili. I tre livelli sono:

Il **frontend** è responsabile dell'interfaccia utente e dell'interazione con l'utente finale. È realizzato con React (JavaScript/JSX), gestito tramite Vite per il build e lo sviluppo locale. Sono utilizzate librerie come react-router-dom per la navigazione, e la WebSocket API nativa per la comunicazione in tempo reale. Lo stile è gestito con CSS e moduli come App.css.

Il **backend** gestisce la logica applicativa, la sicurezza e l'orchestrazione delle operazioni. È sviluppato in Rust, con il framework Axum per la gestione delle richieste HTTP e WebSocket. Utilizza SQLx per l'accesso asincrono al database, Tokio per la concorrenza e la gestione degli eventi, Serde per la serializzazione/deserializzazione dei dati, e Tracing per il logging strutturato. La sicurezza è garantita tramite Argon2 per l'hashing delle password e jsonwebtoken per la generazione dei token JWT.

Il **database** si occupa della persistenza e della gestione dei dati. In fase di sviluppo viene utilizzato SQLite per semplicità e portabilità, mentre la struttura è compatibile con PostgreSQL per ambienti di produzione. Le migrazioni sono gestite tramite script SQL versionati. Le query sono parametrizzate e sicure contro SQL injection.

Inoltre, per lo sviluppo e la gestione del progetto vengono utilizzati diversi tool: Git per il versionamento del codice e la collaborazione, Cargo come gestore di pacchetti e build system per Rust, npm come gestore di pacchetti per il frontend React.

Questa separazione consente di isolare le responsabilità, facilitare l'estensione e la manutenzione del sistema, e garantire una maggiore sicurezza e scalabilità. Nel manuale verranno analizzati in dettaglio ciascuno di questi livelli, illustrando le scelte progettuali, le tecnologie adottate e le possibili evoluzioni future.

Capitolo 2

Analisi schematica dei requisiti

2.1 Obiettivo del sistema

Realizzare un'applicazione client/server per la gestione di una chat testuale, denominata Ruggine, che consenta la comunicazione tra utenti e la creazione di gruppi per la condivisione di messaggi.

2.2 Requisiti funzionali

- **Iscrizione utente**: Al primo avvio, l'utente deve iscriversi tramite una richiesta al server, ottenendo così le credenziali necessarie per accedere al sistema.
- **Login**: L'autenticazione consente all'utente di accedere alle funzionalità della chat, garantendo la sicurezza delle informazioni.
- **Scambio di messaggi**: Il sistema permette l'invio e la ricezione di messaggi testuali in tempo reale tra utenti e gruppi.
- **Creazione gruppi**: Ogni utente ha la possibilità di creare nuovi gruppi di chat, favorendo la comunicazione tematica.
- **Gestione inviti**: L'accesso ai gruppi avviene solo tramite invito; il sistema gestisce l'invio, la ricezione e l'accettazione/rifiuto degli inviti.
- **Compatibilità multipiattaforma**: L'applicazione deve funzionare su almeno due tra Windows, Linux, MacOS, Android, ChromeOS, iOS, garantendo ampia accessibilità.
- **Logging**: Il server produce periodicamente (ogni 2 minuti) dati di log riguardanti la CPU utilizzata, utile per il monitoraggio delle prestazioni.
- **Report dimensione eseguibile**: Nel report finale deve essere inclusa la dimensione del file eseguibile. L'applicazione Ruggine pesa 6.73 MB su sistema operativo Windows 11, mentre pesa 7.9 MB su sistema operativo MacOS Sequoia 15.6.1

2.3 Requisiti non funzionali

- **Prestazioni**: Il sistema è progettato per ridurre il consumo di CPU e la dimensione dell'applicativo, garantendo efficienza anche su dispositivi con risorse limitate.
- **Sicurezza**: L'autenticazione degli utenti e la protezione dei dati sono gestite con tecniche robuste, per prevenire accessi non autorizzati e garantire la privacy.
- **Scalabilità**: L'architettura consente di gestire efficacemente un numero crescente di utenti e gruppi, senza degradare le prestazioni.
- **Usabilità**: L'interfaccia utente è pensata per essere semplice e intuitiva, facilitando l'accesso alle funzionalità principali.
- **Portabilità**: Il codice è strutturato per essere facilmente adattabile alle diverse piattaforme, favorendo la diffusione dell'applicazione.
- **Manutenibilità**: La progettazione modulare facilita l'estensione e la manutenzione futura del sistema.

Capitolo 3

Frontend

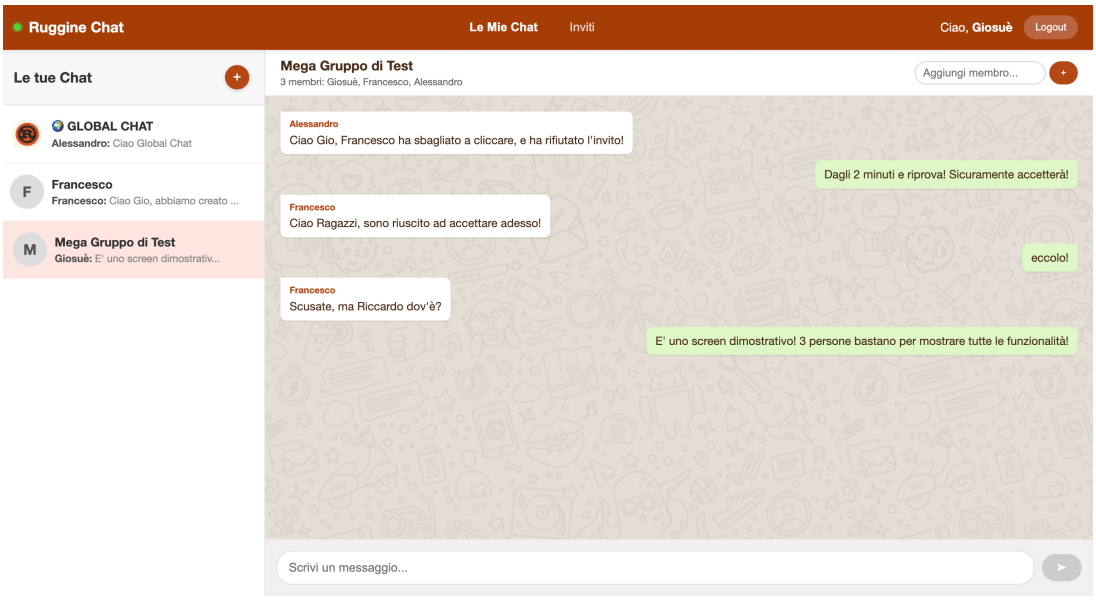
Il frontend di Ruggine è sviluppato in React per garantire una UI moderna, reattiva e facilmente estendibile. La scelta di React è motivata dalla sua ampia diffusione, dalla ricchezza di librerie e dalla possibilità di strutturare l'applicazione in componenti riutilizzabili e testabili.

3.1 Struttura interna

La struttura del frontend è suddivisa in cartelle tematiche. La cartella `components` contiene i componenti riutilizzabili dell'interfaccia, come LoginForm, MessagesContainer, InputArea, Navbar e altri. La cartella `pages` raccoglie le pagine principali dell'applicazione, come LoginPage, ChatPage e InvitesPage, ognuna composta da più componenti. La cartella `context` gestisce lo stato globale tramite React Context, in particolare la connessione WebSocket e l'autenticazione (WebSocketContext.jsx).

3.1.1 ChatPage (ChatPage.jsx)

Pagina principale della chat: sidebar, messaggi e input.



Questa pagina è responsabile del rendering dell'interfaccia principale e della gestione dell'interazione in tempo reale.

Composizione Modulare: La pagina è divisa in componenti specializzati come Sidebar, ChatHeader, MessagesContainer e InputArea. Include anche componenti per il feedback visivo come Toast e Modal per la creazione di nuove chat.

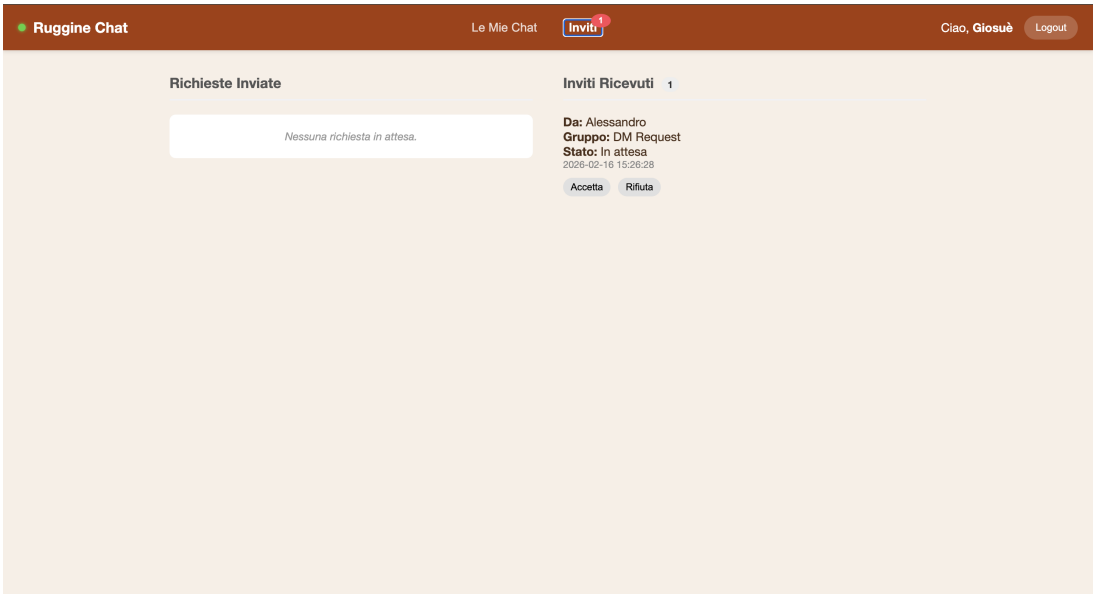
Helper di Formattazione: Contiene funzioni di utilità vitali, come formatGroupName, che intercetta il gruppo con ID 1 rinominandolo in "GLOBAL CHAT", oppure estrae il nome dell'interlocutore corretto nel caso dei Direct Messages (DM).

Auto-Scrolling: Grazie all'uso dell'hook useRef (messagesEndRef) e di useEffect, la vista scorre automaticamente verso il basso ogni volta che arriva un nuovo messaggio o si cambia gruppo.

Selezione Default: Se l'utente non ha selezionato alcun gruppo attivo, la pagina forza la selezione sul gruppo globale (ID 1) come fallback visivo.

3.1.2 InvitesPage (InvitesPage.jsx)

Questa pagina è dedicata esclusivamente alla gestione del ciclo di vita degli inviti (sia per i gruppi che per i DM).



Layout a Colonne: Divide visivamente i dati in due sezioni distinte: una per gli inviti inviati (SentInvitesColumn) e una per quelli ricevuti (ReceivedInvitesColumn).

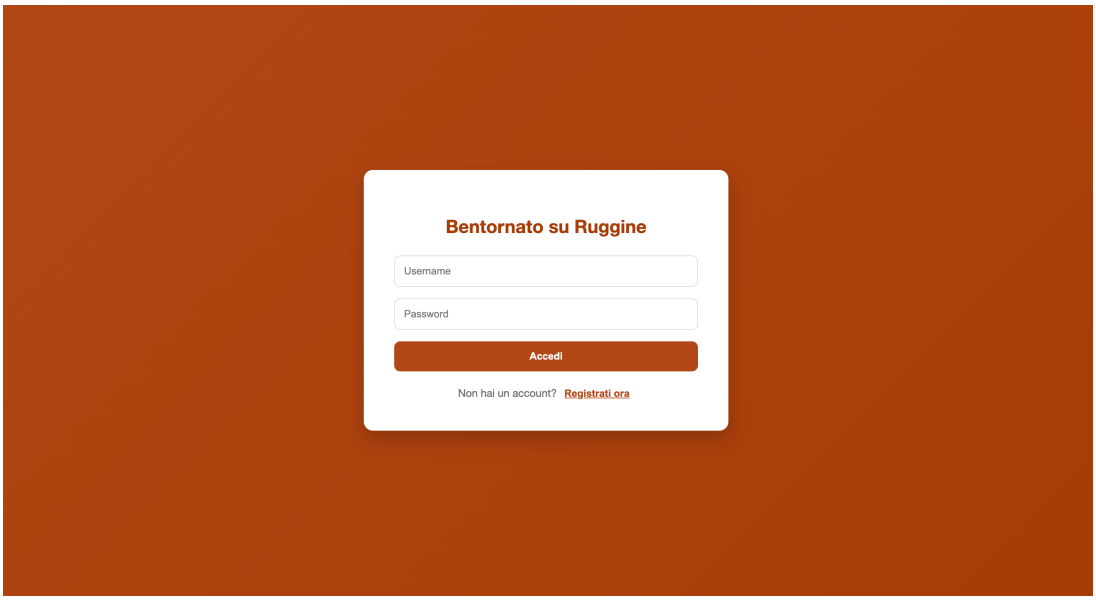
Reattività Temporale (Auto-Refresh): Il componente implementa un meccanismo molto intelligente usando setInterval per forzare un ri-rendering della pagina ogni 5 secondi.

Logica di Scadenza Locale: Utilizzando il timer di cui sopra, la funzione filterActiveInvites calcola dinamicamente la differenza in minuti tra l'ora attuale e l'ora di invio (sent_at).

Questo permette di nascondere in tempo reale gli inviti che sono stati rifiutati da più di 2 minuti, mantenendo l'interfaccia pulita senza dover richiedere continuamente i dati al server.

3.1.3 LoginPage (LoginPage.jsx)

Questa pagina è il punto di partenza dell'utente e funge da ponte tra il protocollo HTTP (stateless) e la successiva connessione WebSocket (stateful).



Modalità Duale: Il componente gestisce un singolo form che si adatta sia per il login che per la registrazione di un nuovo account, alternando lo stato tramite la variabile isLoginMode.

Comunicazione API: A differenza del resto dell'app che vive sul WebSocket, questa pagina effettua chiamate HTTP POST standard (fetch) verso gli endpoint http://127.0.0.1:3000/login e /register del backend.

Gestione della Sessione: Se il login ha successo, il componente estrae il "token" (che il backend ha inviato nel campo message della risposta) e lo salva, insieme all'username, nel sessionStorage del browser. Questo assicura che il token sopravviva ai ricaricamenti della pagina, ma venga eliminato quando si chiude la scheda.

Reindirizzamento: Una volta autenticato, l'utente viene reindirizzato alla pagina principale.

3.2 Comunicazione e flussi

La comunicazione con il backend avviene principalmente tramite API HTTP per operazioni come login, registrazione e invio inviti, e tramite WebSocket per la chat in tempo reale e gli aggiornamenti di gruppo/inviti. Il context globale (WebSocketContext.jsx) incapsula la logica di connessione, gestione messaggi, inviti e autenticazione, esponendo API semplici ai componenti figli.

3.3 Context

Questo file utilizza il pattern Context API di React per mantenere un'unica connessione WebSocket attiva e distribuire i dati (stato globale) a tutte le pagine dell'applicazione senza dover passare continuamente le variabili ("prop drilling").

Ecco come è strutturato e come funziona:

1. Il Riferimento al Socket e lo Stato Globale. Il file utilizza useRef(null) per la variabile socket, il quale permette di mantenere la connessione WebSocket "viva" in background senza causare il ri-rendering dell'intera app ogni volta che arriva un messaggio.

Insieme al socket, il Context mantiene tutti gli stati globali vitali:

user: L'username corrente. groups: L'array delle chat disponibili. messages: Un oggetto dizionario () dove le chiavi sono i group_id e i valori sono array di messaggi. invitations: Un oggetto diviso in due array, received e sent. wsError: Memorizza gli errori per innescare i toast.

2. Auto-Login (Il ciclo di vita). Grazie all'hook useEffect, non appena l'applicazione viene caricata, il Context controlla se ci sono un token e un username salvati nel sessionStorage. Se li trova, imposta l'utente e chiama immediatamente la funzione connect(token) per ripristinare la sessione WebSocket in automatico.

3. La connessione e il "Centralino" (handleMessage). La funzione connect crea l'istanza new WebSocket('ws://127.0.0.1:3000/ws'). Non appena il socket si apre (onopen), invia automaticamente il pacchetto di autenticazione: type: "auth", token .

Quando il server Rust risponde, scatta l'evento onmessage, che passa il JSON alla funzione handleMessage. Questa funzione è uno switch (centralino) che aggiorna lo stato React in base al type del messaggio:

update_group_list: Aggiorna i gruppi e inizializza la cronologia dei messaggi per ogni gruppo. update_invitations_list: Sovrascrive gli array degli inviti inviati e ricevuti. group_chat: Aggiunge il nuovo messaggio all'array corretto all'interno dello stato messages usando il group_id. error: Se l'errore contiene la parola "Token" o "autenticarti", forza il logout automatico dell'utente, altrimenti imposta wsError per mostrare un toast.

4. Le API in Uscita (Trasmettitore). Il Context espone delle comode "funzioni wrapper" per semplificare la vita ai componenti visivi. Invece di far scrivere manualmente il JSON alle varie pagine, fornisce metodi astratti:

sendMessage: Formatta e invia un messaggio di tipo send_group_message. createChat: Formatta dinamicamente un messaggio create_dm o create_group a seconda del tipo richiesto. sendInvite e respondToInvite: Impacchettano i comandi per la gestione degli inviti.

5. L'Esportazione Globale. Alla fine del file, tutti questi stati e queste funzioni vengono inseriti nell'oggetto value e passati al WebSocketContext.Provider. Infine, viene creato l'hook custom useWebSocket. È proprio grazie a questo hook che, ad esempio, ChatPage.jsx può semplicemente richiamare const messages, sendMessage = useWebSocket(); per avere accesso in tempo reale al database della chat e al canale di trasmissione.

3.4 Sicurezza e validazione

- I dati inseriti dall'utente sono validati lato client (es. campi obbligatori, lunghezza minima password). I messaggi tra Client e Server sono forzati come spiegato precedentemente a delle strutture standard.
- I token di autenticazione sono gestiti in sessionStorage, per gestire la sessione ed essere sempre autenticati con il server Rust durante la comunicazione.
- La comunicazione WebSocket è autenticata tramite token.

3.5 Motivazioni progettuali

La struttura a componenti e l'uso di Context rendono il frontend facilmente estendibile e manutenibile. Ogni funzione è isolata e testabile. L'adozione di WebSocket per la chat garantisce reattività e scalabilità. L'approccio "separation of concerns" tra presentazione, logica e stato migliora la qualità del codice e la collaborazione tra sviluppatori.

Capitolo 4

Backend

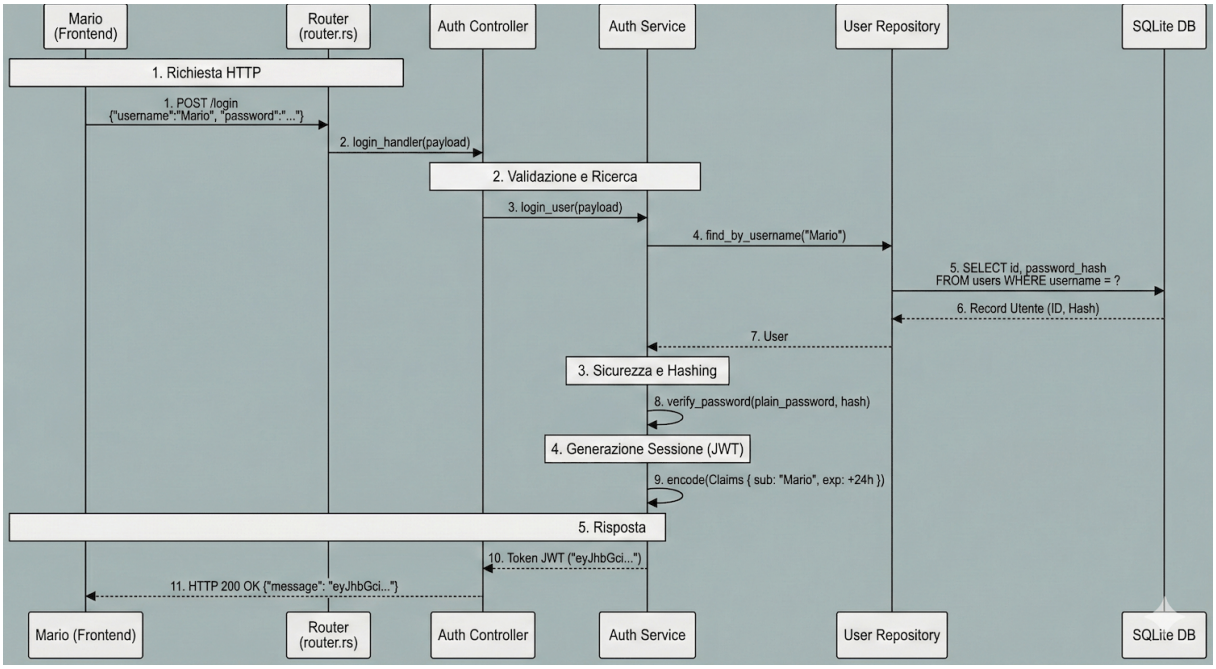
Il backend di Ruggine è sviluppato in Rust per garantire performance, sicurezza e concorrenza. L'adozione di Axum, SQLx e Tokio consente di gestire richieste HTTP/WebSocket in modo asincrono e scalabile. La struttura a livelli separa chiaramente responsabilità e facilita la manutenzione.

4.1 Struttura interna

Il backend segue il pattern controller-service-repository. I controller gestiscono le richieste in ingresso (HTTPWS), validano i dati e orchestrano la logica applicativa. I service implementano la business logic, come validazione, hashing password, generazione token, gestione gruppi. I repository si occupano dell'accesso ai dati, eseguendo query sicure e ottimizzate.

4.2 Flusso Logico del login

Immaginiamo che l'utente "Mario" inserisca le sue credenziali e prema il pulsante "Accedi" sul frontend.



Fase 1: La Richiesta HTTP (Router e Controller). Arrivo della Richiesta: Il frontend invia una richiesta HTTP POST all'endpoint /login. Il payload della richiesta è un JSON strutturato secondo il DTO AuthPayload, contenente username e password.

Instradamento: Il file router.rs intercetta la richiesta e la indirizza alla funzione login_handler all'interno dell' authentication_controller.rs.

Estrazione Dati: Axum (il framework web) estrae automaticamente il JSON dal corpo della richiesta e lo trasforma nella struct Rust AuthPayload. Il controller passa quindi questo payload e il riferimento al database alla logica di business: auth_service::login_user.

Fase 2: Validazione e Ricerca (Service e Repository). Validazione Input: All'interno di auth_service.rs, il sistema verifica per prima cosa che l'username e la password non siano stringhe vuote. Se lo sono, blocca subito l'operazione restituendo un AppError::ValidationError.

Interrogazione del Database: Il servizio chiama user_repository::find_by_username.

Esecuzione Query: Il repository esegue in modo sicuro la query SQL

```
SELECT id, username, password_hash FROM users WHERE username = ?
```

Utente non trovato: Se la query non restituisce alcun risultato, il servizio interrompe il flusso e restituisce l'errore generico "Invalid username or password".

Fase 3: Sicurezza e Hashing (Service). Verifica della Password: Se l'utente "Mario" viene trovato, il database restituisce i suoi dati, incluso il password_hash. Il servizio chiama la funzione verify_password.

Confronto Argon2: Questa funzione utilizza l'algoritmo Argon2 per processare la password in chiaro inviata da Mario e confrontarla matematicamente con l'hash salvato nel database. Se non combaciano, viene restituito nuovamente l'errore "Invalid username or password." per non dare indizi a potenziali malintenzionati.

Fase 4: Generazione della Sessione (JWT). Creazione dei Claims: Le credenziali sono corrette. Il servizio prepara i dati da inserire nel token JWT (chiamati Claims): l'username (sub), l'ID utente (user_id) e la data di scadenza (exp), impostata esattamente a 24 ore dal momento del login.

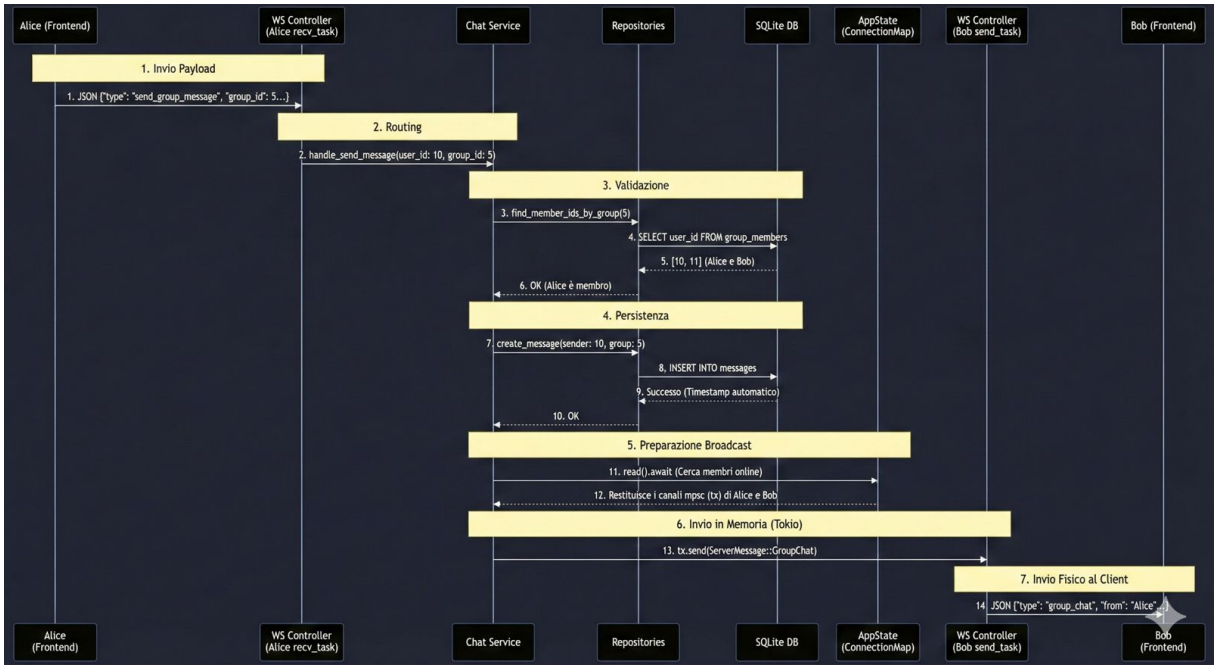
Firma del Token: Il token viene codificato e firmato crittograficamente usando una **chiave segreta fissa** del server (che in teoria dovrebbe stare nei file .env e **non essere mai divulgata**) per garantirne l'inviolabilità.

Tutta questa operazione è gestita tramite l'utilizzo della libreria esterna "jsonwebtoken".

Fase 5: La Risposta (Controller). Ritorno al Client: Il token generato viene restituito al login_handler. Il controller lo inserisce nel campo message della struttura DTO AuthResponse.

Consegna: Axum converte l' AuthResponse in JSON e invia una risposta HTTP 200 OK al frontend. Da questo momento in poi, Mario userà questo token per aprire la connessione WebSocket.

4.3 Flusso Logico dei messaggi



Scenario: Mittente: Alice (ID: 10, Username: "Alice")

Destinazione: Gruppo "Progetto Rust" (ID: 5)

Messaggio: "Ciao a tutti!"

Utenti Online nel gruppo: Alice (ID: 10), Bob (ID: 11).

Flusso all'interno del backend Ruggine:

Fase 1: Il Frontend invia il Payload (WebSocket). Il frontend di Alice usa la connessione WebSocket aperta per inviare una stringa JSON al server. Grazie al ClientMessage definito nei DTO, il JSON sarà strutturato così:

```
{
  "type": "send_group_message",
  "group_id": 5,
  "content": "Ciao a tutti!"
}
```

Fase 2: Il Controller riceve e smista (web_socket_controller.rs). Il recv_task del socket di Alice, in ascolto continuo, intercetta il messaggio di testo. Usa serde_json per deserializzare la stringa e capisce (tramite il tag "type") che corrisponde all'enum

`ClientMessage::SendGroupMessage.`

Il controller verifica che Alice sia autenticata (la variabile user_info contiene i suoi dati). Il costrutto match instrada i dati chiamando la logica di business:

`chat_service::handle_send_message(..., 10, "Alice", 5, "Ciao a tutti!").`

Fase 3: Validazione e Sicurezza (chat_service.rs).

Prima di fare qualsiasi cosa, il chat_service deve assicurarsi che Alice abbia il diritto di scrivere in quella chat. Chiama group_repository::find_member_ids_by_group passando il database e l'ID del gruppo 5.

Controlla se l'ID di Alice (10) è presente nella lista restituita. Se Alice stesse cercando di "hackerare" il sistema inviando un ID gruppo a cui non appartiene, il servizio bloccherebbe l'operazione restituendo un

`AppError::ValidationError("Non sei membro di questo gruppo.").`

Fase 4: Salvataggio nel Database (message_repository.rs). La validazione è passata. Ora bisogna rendere il messaggio permanente. Il servizio chiama

`message_repository::create_message.`

Il repository esegue la query SQL

```
INSERT INTO messages (group_id, sender_user_id, content) VALUES (?, ?, ?).
```

SQLite salva il record e, grazie alla definizione della tabella nel file di migrazione, aggiunge automaticamente il timestamp esatto nel campo created_at.

Fase 5: Preparazione del Broadcast (chat_service.rs). Il messaggio è salvo nel DB, ma Bob (che è online) non lo sa ancora. Bisogna avvisarlo.

Il chat_service costruisce il pacchetto di uscita, ovvero un ServerMessage::GroupChat contenente l'ID del gruppo 5, il nome "Alice" e il testo "Ciao a tutti!".

Chiama la funzione asincrona interna broadcast_to_group_members. La prima attività di questa funzione è trovare tutti gli ID dei partecipanti al gruppo di interesse nel DB.

Fase 6: Ricerca dei Socket e Invio Interno (Stato Globale). All'interno di broadcast_to_group_members usiamo le funzionalità offerte dalla libreria Tokyo per la concorrenza:

Il server acquisisce un blocco di lettura (read()) sulla mappa globale delle connessioni ConnectionMap.

Scorre la lista degli ID dei membri del gruppo 5 (ovvero 10 e 11). Cerca l'ID 11 (Bob) nella mappa. Bob è online, quindi il server trova il suo o i suoi "trasmettitori" (i canali tx di Tokio associati ai suoi socket).

Il server invia il ServerMessage in memoria, infilandolo nel canale tx di Bob.

Se Bob non fosse online, il messaggio sarebbe comunque salvato nel DB, e sarebbe consegnato a Bob nel momento in cui ritornasse online.

Nota tecnica: Il server invia il messaggio anche nel canale tx di Alice (10). Questo è un pattern utilissimo: il frontend di Alice capirà che il server ha elaborato il messaggio con successo quando lo riceverà indietro.

Fase 7: Invio Fisico al Frontend di Bob (we_socket_controller.rs). Spostiamoci ora nel task parallelo che gestisce il socket di Bob.

Il send_task di Bob, che era in attesa silenziosa, vede apparire il messaggio nel suo canale rx.

Lo serializza di nuovo in formato JSON. Lo spara attraverso il cavo di rete usando sender.send(Message::Text(...)). Fine del ciclo Il frontend di Bob riceve questo JSON:

```
{
  "type": "group_chat",
  "group_id": 5,
  "from_username": "Alice",
  "content": "Ciao a tutti!"
}
```

React legge il JSON, aggiorna lo stato dell'applicazione e fa comparire il fumetto con il messaggio di Alice sullo schermo di Bob in tempo reale. Tutto questo processo, in Rust, avviene in una frazione di millisecondo.

4.4 Main e Entry Point

4.4.1 Main (main.rs)

Entry point del server Rust "Ruggine" Avvia il server, gestisce setup logging, database, stato globale e task periodici.

4.4.2 CPU logger (cpu_logger.rs)

Log periodico CPU server (cross-platform). Il file utilizza la compilazione condizionale

```
#[cfg(target_os = "...")]
```

per implementare la funzione get_cpu_time() in modo specifico per ogni sistema operativo.

La funzione principale è spawn_cpu_logger(period_secs: u64), che viene lanciata dal main.rs all'avvio del server. Asincronia: Utilizza tokio::spawn per creare un task separato che gira in background senza bloccare il traffico della chat. Timer: Un interval di Tokio scandisce il ritmo (ogni 120 secondi, come configurato nel main.rs).

state.rs error.rs server_cpu_stats.log cpu_logger.rs main.rs

```
1  [2026-02-16 17:03:33] Server CPU Usage: 0.16387% | CPU Time Usage: 0.010s
2  [2026-02-16 17:05:33] Server CPU Usage: 0.71490% | CPU Time Usage: 0.863s
3  [2026-02-16 17:07:33] Server CPU Usage: 0.36340% | CPU Time Usage: 0.434s
4  [2026-02-16 17:09:33] Server CPU Usage: 0.03788% | CPU Time Usage: 0.046s
5  [2026-02-16 17:11:33] Server CPU Usage: 0.03928% | CPU Time Usage: 0.037s
6  [2026-02-16 17:12:22] Server CPU Usage: 0.08615% | CPU Time Usage: 0.006s
7  [2026-02-16 17:14:22] Server CPU Usage: 0.03654% | CPU Time Usage: 0.056s
```

4.4.3 Error (error.rs)

Gestione centralizzata degli errori custom per il backend Ruggine. Definisce l'enum AppError e le conversioni/risposte per errori comuni (DB, hashing, validazione)

```rust
#[derive(Debug)]
pub enum AppError {
    SqlxError(sqlx::Error),
    HashError(password_hash::Error),
    ValidationError(String),
}
```

4.4.4 Router (router.rs)

Definizione delle rotte HTTP e WebSocket dell'applicazione. La funzione create_router() restituisce un oggetto Router, che è il componente di Axum incaricato di gestire il traffico. Il router collega un metodo HTTP (come GET o POST) e un percorso (path) a una funzione specifica (Handler).

```rust
pub fn create_router() -> Router<Arc<AppState>> {
    Router::new()
        // Rotte di autenticazione
        .route("/register", post(authentication_controller::register_handler))
        .route("/login", post(authentication_controller::login_handler))
        // Rotta WebSocket
        .route("/ws", get(web_socket_controller::ws_handler))
}
```

4.4.5 State (state.rs)

Stato globale condiviso per il backend Ruggine. Definisce la struttura AppState, che contiene il pool di connessioni al database (db) e la mappa dei WebSocket attivi (connections).

```
pub type ConnectionMap = Arc<RwLock<HashMap<
i64,HashMap<Uuid, mpsc::Sender<ServerMessage>>>>>;

pub struct AppState {
    pub db: SqlitePool,
    pub connections: ConnectionMap,
}
```

Ogni singola finestra del browser o dispositivo connesso riceve un identificativo univoco (UUID), mentre i64 rappresenta l'ID univoco dell'utente nel database. Un utente potrebbe connettersi contemporaneamente da PC e da smartphone. Questa mappa permette al server di inviare il messaggio a tutte le sessioni aperte di quell'utente.

4.5 Controllers

Il loro compito principale è ricevere i dati, convalidarli a livello superficiale e smistarli ai servizi corretti.

4.5.1 Authentication Controller (authentication_controller.rs)

Gestione delle richieste di autenticazione. Controller per registrazione e login utente, interfaccia con auth_service.

- **register_handler**

  ```
  pub async fn register_handler(
  State(state): State<Arc<AppState>>,
  Json(payload): Json<AuthPayload>,
  ) -> Result<Json<AuthResponse>, AppError>
  ```

  Controller per la registrazione di un nuovo utente

- **login_handler**

  ```
  pub async fn login_handler(
  State(state): State<Arc<AppState>>,
  Json(payload): Json<AuthPayload>,
  ) -> Result<Json<AuthResponse>, AppError>
  ```

  Controller per il login di un utente, ritorna il JWT in caso di successo. Questo token verrà utilizzato in tutte le successive richieste dell'utente

4.5.2 Web socket Controller (web_socket_controller.rs)

Gestione delle connessioni e messaggi WebSocket lato server. Implementa handler Axum per upgrade, ricezione e invio messaggi, gestione utenti connessi.

- **ws_handler**

```
pub async fn ws_handler(
ws: WebSocketUpgrade,
State(state): State<Arc<AppState>>,
) -> impl IntoResponse
```

Handler Axum per l'upgrade della connessione HTTP a WebSocket

- **handle_socket**

```
async fn handle_socket(
socket: WebSocket,
state: Arc<AppState>
)
```

Gestisce la comunicazione asincrona su un socket WebSocket. Il protocollo WebSocket è full-duplex (lettura e scrittura avvengono contemporaneamente).

Il canale interno (mpsc::channel): Viene creato un canale Tokio con un buffer di 100 messaggi. Il tx (trasmettitore) verrà usato dalla logica di business per "mettere in coda" i messaggi da inviare al client, mentre il rx (ricevitore) verrà ascoltato dal task di invio.

```
let (tx, mut rx) = mpsc::channel::<ServerMessage>(100);
```

Il Task di Invio (send_task). Ascolta continuamente il canale rx. Quando un servizio nel backend invia un ServerMessage sul tx, il send_task lo intercetta.

```
let mut send_task = tokio::spawn(async move {...
```

Il Task di Ricezione e la Logica di Business (recv_task). Deserializzazione e Pattern Matching: Ogni messaggio di testo in arrivo viene convertito in un enum ClientMessage. Se il JSON è malformato, logga l'errore e passa al prossimo messaggio ignorandolo (continue).

```
let mut recv_task = tokio::spawn(async move {...
```

Il "Guard" di Autenticazione: C'è un blocco if cruciale. Se user_info è vuoto (utente non autenticato) e il messaggio non è una richiesta di Auth, il server blocca l'azione, restituisce un errore (ServerMessage::Error) e non processa il comando.

Routing dei Comandi (match msg):

Auth: È il payload più importante. Se il token è valido, salva le informazioni dell'utente, inietta il tx (il canale per parlargli) nella ConnectionMap globale dello AppState, e gli invia i dati iniziali (lista gruppi e inviti).

```
match msg {
   ClientMessage::Auth { token } => {
       match user_service::authenticate_ws_user
       (&state_c.db, &token).await {...
```

Altri comandi (SendGroupMessage, CreateDM, ecc.): Vengono instradati verso i rispettivi metodi del chat_service.

```
ClientMessage::SendGroupMessage { group_id, content } => {...
```

Gestione Centralizzata degli Errori: Se un servizio restituisce un AppError, questo viene intercettato, viene loggato il warning nel server e si invia l'errore al client tramite WebSocket.

4.6 Services

Contengono la logica di business: regole, validazioni e coordinamento delle operazioni.

4.6.1 User Service (user_service.rs)

Gestione utenti connessi via WebSocket e autenticazione JWT, strutture e funzioni per sessione utente e validazione token

- **authenticate_ws_user**

```
pub async fn authenticate_ws_user(
_db: &SqlitePool,
token: &str
) -> Result<WsUser, AppError>
```

Autentica un utente tramite il suo token (username) e restituisce i suoi dati di sessione

4.6.2 Chat Service (chat_service.rs)

Logica di business per la chat, broadcast e gestione gruppi. Funzioni di invio messaggi, broadcast, gestione gruppi e interazione con repository

- **broadcast_to_group_members**

```
async fn broadcast_to_group_members(
db: &SqlitePool,
connections: &ConnectionMap,
group_id: i64,
message: ServerMessage,
)
```

Invia un messaggio a tutti i membri di un gruppo

- **broadcast_message_to_user**

```
async fn broadcast_message_to_user(
connections: &ConnectionMap,
user_id: i64,
message: ServerMessage,
)
```

Invia un messaggio a un singolo utente

- **refresh_invitations_for_user**

```
pub async fn refresh_invitations_for_user(
state: Arc<AppState>,
user_id: i64,
```

```
)
```

Helper per inviare la lista aggiornata degli inviti a un utente

- **get_user_data_for_client**

```
pub async fn get_user_data_for_client(
db: &Pool<Sqlite>,
user_id: i64,
) -> Result<Vec<GroupInfo>, AppError>
```

Recupera tutti i dati necessari per il client di un utente

- **handle_send_message**

```
pub async fn handle_send_message(
state: Arc<AppState>,
sender_user_id: i64,
sender_username: &str,
group_id: i64,
content: &str,
) -> Result<(), AppError>
```

Gestisce l'invio di un messaggio in un gruppo

- **handle_create_group**

```
pub async fn handle_create_group(
db: &Pool<Sqlite>,
creator_user_id: i64,
group_name: &str,
) -> Result<Vec<GroupInfo>, AppError>
```

Gestisce la richiesta di creazione di un nuovo gruppo

- **handle_create_dm**

```
pub async fn handle_create_dm(
state: Arc<AppState>,
creator_user_id: i64,
target_username: &str,
) -> Result<(), AppError>
```

Gestisce la richiesta di creazione DM (invia invito)

- **handle_invite_to_group**

```
pub async fn handle_invite_to_group(
state: Arc<AppState>,
inviter_username: &str,
group_id: i64,
target_username: &str,
) -> Result<(), AppError>
```

Gestisce l'invito di un utente a un gruppo

- **handle_invitation_response**

```
    pub async fn handle_invitation_response(
    state: Arc<AppState>,
    user_id: i64,
    invitation_id: i64,
    accept: bool,
) -> Result<(), AppError>
```

Gestisce la risposta a un invito (accetta/rifiuta)

4.6.3 Authentication Service (auth_service.rs)

Logica di business per autenticazione e registrazione utenti. Funzioni di validazione, hashing, JWT e interazione con repository utenti/gruppi

- **register_user**

```
    pub async fn register_user(
    db: &SqlitePool,
    payload: AuthPayload
) -> Result<(), AppError>
```

Logica di business per la registrazione di un nuovo utente.

- **login_user**

```
    pub async fn login_user(
    db: &SqlitePool,
    payload: AuthPayload
) -> Result<String, AppError>
```

Logica di business per il login di un utente

- **hash_password**

```
    pub fn hash_password(
    password: &str
) -> Result<String, password_hash::Error>
```

Trasforma una password in chiaro in una stringa sicura (hash) prima che venga salvata nel database.

- **verify_password**

```
    pub fn verify_password(
    password: &str,
    hash: &str
) -> Result<bool, password_hash::Error>
```

Utilizzata durante il login per verificare se la password inserita dall'utente corrisponde a quella memorizzata

4.7 Repositories

Questa sezione descrive l'architettura dei repository, responsabili dell'interfaccia diretta con il database **SQLite** tramite la libreria asincrona **SQLx**.

L'obiettivo è isolare la logica SQL dal resto dell'applicazione garantendo atomicità nelle operazioni complesse e integrità referenziale.

4.7.1 Group Repository (group_repository.rs)

Questo modulo gestisce il ciclo di vita dei gruppi e delle relazioni di appartenenza.

- **create_group**

```
pub async fn create_group(
    tx: &mut Transaction<'_, Sqlite>, // Richiede una transazione attiva
    name: Option<&str>,
    is_dm: bool,
) -> Result<i64, AppError>
```

  Inserisce un nuovo gruppo (DM o standard) nel database. Restituisce l'ID creato. Per transazione attiva si intende un interazione con il DB che abbia le proprietà ACID.

- **add_member**

```
pub async fn add_member(
tx: &mut Transaction<'_, Sqlite>, // Richiede una transazione attiva
group_id: i64,
user_id: i64,
) -> Result<(), AppError>
```

  Aggiunge un singolo utente a un gruppo (usato per il creatore).

- **add_member_or_ignore**

```
pub async fn add_member_or_ignore(
db: &Pool<Sqlite>, // Usa il pool principale
group_id: i64,
user_id: i64,
) -> Result<(), AppError>
```

  Aggiunge un utente a un gruppo, ignorando l'errore se è già membro. Usato per la logica di 'InviteToGroup'.

- **find_by_group_id**

```
pub async fn find_group_by_id(
db: &Pool<Sqlite>,
group_id: i64,
) -> Result<Option<Group>, AppError>
```

  Trova i dettagli di un singolo gruppo (come 'is_dm').

- **find_group_ids_by_user**

```
pub async fn find_group_ids_by_user(
db: &Pool<Sqlite>,
user_id: i64,
) -> Result<Vec<i64>, AppError>
```

Recupera tutti gli ID dei gruppi a cui un utente appartiene.

- **find_members_by_group**

```
pub async fn find_members_by_group(
db: &Pool<Sqlite>,
group_id: i64,
) -> Result<Vec<GroupMember>, AppError>
```

Recupera tutti i membri (con ID e Username) di un dato gruppo.

- **find_member_ids_by_group**

```
pub async fn find_member_ids_by_group(
db: &Pool<Sqlite>,
group_id: i64,
) -> Result<Vec<i64>, AppError>
```

Recupera solo gli ID di tutti i membri di un dato gruppo. Molto efficiente per la logica di broadcast.

- **find_existing_dm_between_users**

```
pub async fn find_existing_dm_between_users(
db: &Pool<Sqlite>,
user_id_1: i64,
user_id_2: i64,
) -> Result<Option<i64>, AppError>
```

Cerca se esiste già un gruppo DM attivo tra due utenti.

4.7.2 Invitation Repository (invitation_repository.rs)

Funzioni CRUD per inviti, transazioni e query correlate.

- **create_invitation**

```
pub async fn create_invitation(
db: &Pool<Sqlite>,
from_user_id: i64,
to_user_id: i64,
group_id: i64,
) -> Result<i64, AppError>
```

Crea un nuovo invito (versione standard Gruppi)

- **create_invitation_tx**

```
pub async fn create_invitation_tx(
tx: &mut Transaction<'_, Sqlite>,
from_user_id: i64,
to_user_id: i64,
group_id: i64,
) -> Result<i64, AppError>
```

Versione transazionale di create_invitation. Fondamentale per la creazione atomica dei DM (Gruppo + Membro + Invito).

- **update_status**

```
    pub async fn update_status(
    db: &Pool<Sqlite>,
    invitation_id: i64,
    new_status: &str,
) -> Result<(), AppError>
```

Aggiorna lo stato di un invito (accetta/rifiuta).

- **find_received_invites**

```
    pub async fn find_received_invites(
    db: &Pool<Sqlite>,
    user_id: i64,
) -> Result<Vec<InvitationInfo>, AppError>
```

Inviti ricevuti (partner = chi mi ha invitato)

- **find_sent_invites**

```
    pub async fn find_sent_invites(
    db: &Pool<Sqlite>,
    user_id: i64,
) -> Result<Vec<InvitationInfo>, AppError>
```

Inviti inviati (partner = chi ho invitato)

- **find_by_id**

```
pub async fn find_by_id(
    db: &Pool<Sqlite>,
    invitation_id: i64,
) -> Result<Option<Invitation>, AppError>
```

Recupera un invito per ID.

- **find_pending_dm_invitation**

```
pub async fn find_pending_dm_invitation(
    db: &Pool<Sqlite>,
    user_a: i64,
    user_b: i64,
) -> Result<Option<Invitation>, AppError>
```

Cerca se esiste già un invito DM pendente tra due utenti.

- **is_in_cooldown_dm**

```
    pub async fn is_in_cooldown_dm(
    db: &Pool<Sqlite>,
    user_a: i64,
    user_b: i64,
) -> Result<bool, AppError>
```

Verifica se esiste un invito DM rifiutato tra due utenti negli ultimi 2 minuti.

- **check_group_invitation_status**

```
pub async fn check_group_invitation_status(
    db: &Pool<Sqlite>,
    from_user: i64,
    to_user: i64,
    group_id: i64,
) -> Result<String, AppError>
```

Controlla lo stato di un invito di gruppo tra due utenti.

4.7.3 Message Repository (message_repository.rs)

Funzioni per salvataggio, recupero e formattazione messaggio.

- **create_message**

```
pub async fn create_message(
    db: &Pool<Sqlite>,
    group_id: i64,
    sender_user_id: i64,
    content: &str,
) -> Result<(), AppError>
```

Salva un singolo messaggio di chat nel database (operazione atomica)

- **get_history_for_group**

```
pub async fn get_history_for_group(
    db: &Pool<Sqlite>,
    group_id: i64,
) -> Result<Vec<ChatMessage>, AppError>
```

Recupera gli ultimi 50 messaggi per un dato gruppo o DM (cronologia)

4.7.4 User Repository (user_repository.rs)

Funzioni CRUD per utente e query correlate

- **create_user**

```
pub async fn create_user(
    db: &SqlitePool,
    username: &str,
    hash: &str
) -> Result<(), AppError>
```

Crea un nuovo utente nel database

- **find_by_username**

```
pub async fn find_by_username(
    db: &Pool<Sqlite>,
    username: &str
```

```
) -> Result<Option<User>, AppError>
```

Trova un utente per username

Capitolo 5

Database

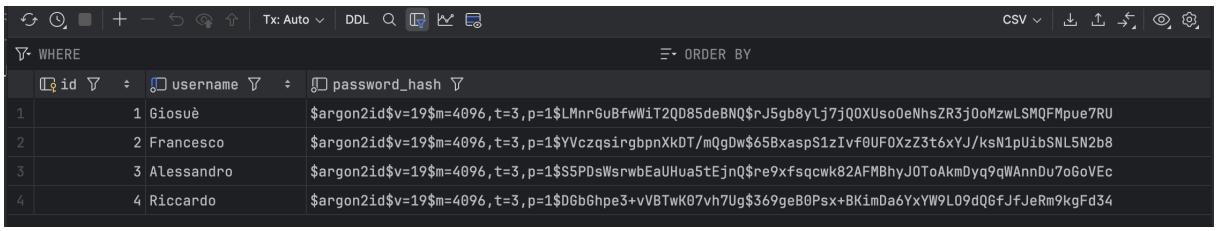
Il database è il cuore della persistenza dati di Ruggine. È progettato per garantire integrità, coerenza e performance, supportando autenticazione, chat, gruppi e inviti. La progettazione segue principi di normalizzazione e sicurezza.

5.1 Struttura e tabelle principali

Le principali tabelle del database sono:

- `users`
- `groups`
- `group_members`
- `messages`
- `invitations`

5.1.1 users

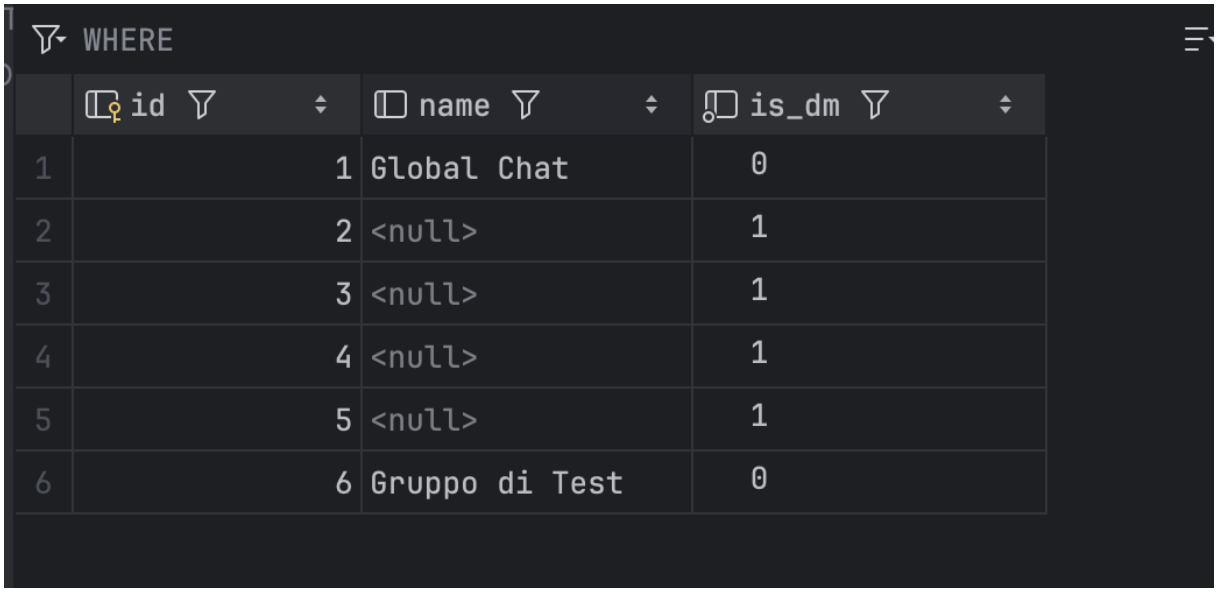


Memorizza l'identità e le credenziali degli utenti

- `id`: Chiave primaria autoincrementale
- `username`: Testo obbligatorio e univoco, utilizzato per l'identificazione e la generazione dei Token JWT
- `password_hash`: Stringa contenente l'hash della password generato tramite Argon2. Il formato salvato di Argon è:

```
$<algoritmo>$v=<versione>$<parametri>$<salt_base64>$<hash_base64>
```

5.1.2 groups



Definisce Gruppi e chat private (DM).

- `id`: Chiave primaria autoincrementale
- `name`: Opzionale, contiene il nome delle chat di gruppo, o NULL per le chat DM
- `is_dm`: Booleano che distingue tra gruppi standard e DM

Nota: La tabella contiene il record statico: **GLOBAL CHAT** (ID: 1)

5.1.3 group_members

WHERE

	group_id	user_id
1	1	1
2	1	2
3	2	1
4	2	2
5	1	3
6	3	3
7	4	3
8	4	1
9	5	3
10	1	4

Gestisce la relazione Many-to-Many tra Utenti e Gruppi

- `group_id & user_id`: Chiave primaria composta

5.1.4 messages

WHERE ORDER BY

	id	group_id	sender_user_id	content	created_at
1	1	2	1	Ciao!	2026-02-16 18:16:25
2	2	1	2	Ciao Chat Globale	2026-02-16 18:17:53
3	3	1	1	ciao Fra	2026-02-16 18:18:01
4	4	6	3	messaggio di prova	2026-02-16 18:19:25
5	5	6	4	gruppo test	2026-02-16 18:19:30

Archivia tutti i messaggi scambiati.

- `id`: Chiave primaria autoincrementale
- `group_id`: Identifica il gruppo in cui è stato mandato il messaggio (anche DM).

- `sender_user_id`: Riferimento all'id del mittente
- `content`: Testo del messaggio
- `created_at`: Timestamp automatico impostato all'ora locale del server al momento dell'inserimento

5.1.5 invitations

WHERE ORDER BY

	user_id	to_user_id	group_id	status	created_at	updated_at
1	1	2	2	accepted	2026-02-16 17:04:47	2026-02-16 17:05:14
2	3	1	3	rejected	2026-02-16 17:06:05	2026-02-16 17:06:09
3	3	1	4	accepted	2026-02-16 17:10:06	2026-02-16 17:10:16
4	3	2	5	rejected	2026-02-16 17:12:55	2026-02-16 17:12:57
5	4	1	6	accepted	2026-02-16 18:18:48	2026-02-16 18:19:00
6	4	2	6	rejected	2026-02-16 18:18:52	2026-02-16 18:19:03
7	4	3	6	accepted	2026-02-16 18:18:55	2026-02-16 18:19:20

Gestisce le richieste di accesso ai gruppi o di creazione DM

- `id`: Chiave primaria
- `from_user_id`: Riferimento all'id del mittente
- `to_user_id`: Riferimento all'id del destinatario
- `group_id`: Riferimento all'id del Gruppo o DM
- `status`: Stato dell'invito (pending, accepted, rejected)
- `created_at`: Timestamp automatico impostato all'ora locale del server al momento dell'inserimento
- `updated_at`: Timestamp automatico impostato all'ora locale del server al momento dell'ultima modifica

5.2 Tecnologie e motivazioni

Il database scelto è SQLite per semplicità e portabilità in fase di sviluppo, ma la struttura è compatibile con PostgreSQL per ambienti di produzione. L'accesso avviene tramite SQLx, che garantisce query asincrone e sicure. La scelta di SQLx e delle migrazioni versionate garantisce riproducibilità e sicurezza. L'architettura a tabelle separate isola le responsabilità e semplifica la manutenzione.

5.3 Sicurezza e integrità

- Vincoli di chiave esterna e ON DELETE CASCADE per coerenza referenziale.
- Query parametrizzate per prevenire SQL injection.

- Migrazioni versionate per tracciare ogni modifica allo schema.
- Trigger per aggiornare automaticamente i timestamp.

Capitolo 6

Proposte di migliorie e feature future

Questa sezione presenta proposte concrete per l'evoluzione di Ruggine, motivate da esigenze di scalabilità, robustezza e competitività.

6.1 Migliorie architetturali

Tra le principali proposte architetturali vi è l'adozione di microservizi, ad esempio separando autenticazione, chat e notifiche in servizi distinti. Questo permette di aggiornare la chat senza toccare l'autenticazione, o di scalare solo il servizio più usato: un picco di traffico nella chat non rallenta il login degli utenti. La containerizzazione con Docker consente di avviare ogni servizio (backend, frontend, database) in un container isolato; lo sviluppatore può lanciare l'intero sistema con un solo comando (docker-compose up) senza preoccuparsi delle dipendenze locali. Infine, l'integrazione di CI/CD permette di automatizzare test, build e deploy: ogni modifica al backend viene testata e, se supera i test, viene automaticamente deployata in staging.

6.2 Nuove feature

Tra le nuove feature proposte vi sono le notifiche push, che permettono a un utente di ricevere una notifica sul telefono quando viene invitato in un gruppo o riceve un nuovo messaggio (implementazione possibile tramite Firebase Cloud Messaging). Il supporto mobile può essere realizzato tramite una Progressive Web App, che consente l'uso offline e l'installazione su smartphone senza passare dagli store, oppure tramite app native che sfruttano notifiche e funzionalità avanzate del dispositivo. La gestione avanzata dei gruppi prevede l'aggiunta di ruoli (admin, moderatore, membro), permessi granulari (ad esempio solo l'admin può invitare) e strumenti di moderazione (ban, silenzia) per comunità più grandi e sicure. L'integrazione con servizi esterni può includere il login tramite Google (OAuth), il salvataggio file su cloud (ad esempio Google Drive) e la raccolta di statistiche di utilizzo tramite Google Analytics.

6.3 Riflessioni su Token e Sicurezza

Il sistema di autenticazione dell' applicazione utilizza i JSON Web Tokens (JWT) per gestire le sessioni in modo sicuro e senza stato (stateless).

Il token scambiato nei messaggi è in chiaro.

Un JWT è composto da tre parti codificate in Base64Url: Header, Payload e Signature.

I dati sono leggibili: Chiunque entri in possesso del token può decodificare il Payload (i parametri sopra elencati) usando strumenti online. Non devi mai inserire password o dati sensibili nel token.

La firma è protetta: La terza parte del token (Signature) è generata usando una chiave segreta nota solo al server. Se un malintenzionato modifica anche solo un carattere dell'username nel payload, la firma non corrisponderà più e il server rifiuterà il token.

Tuttavia con l'attuale implementazione è possibile fingersi qualcun altro se si prendesse (in maniera illecita) il suo token.

Se un malintenzionato riuscisse a rubare un token attivo (ad esempio tramite un attacco Cross-Site Scripting o intercettando il traffico se non criptato), potrebbe inviarlo al server e questo lo riconoscerebbe come l'utente legittimo finché il token non scade.

Questo fenomeno si chiama Token Theft.

Comunque, il malintenzionato non può "creare" un nuovo token falso (ad esempio cambiare l'ID utente da 10 a 1) perché non possiede la chiave segreta necessaria per generare una firma valida.

Per i motivi precedentemente citati, è fondamentale che i token siano scambiati solo su connessioni sicure (HTTPS/WSS). L'ideale sarebbe avere delle connessioni protette con tutti gli utenti collegati al server, sfruttando il TLS. Per avere HTTPS e connessioni protette da TLS servirebbe avere un Certificato ufficiale rilasciato da una Certification Authority (CA).